

Question No:3

DS	Session	Queue	Question Information	Level 1	Challenge 53
Question description					
Ramesh is an DS expert training youngsters struggling in DS to make them better.					
Ramesh usually gives interesting problems to the youngsters to make them love the DS.					
One such day Sathya provided to the youngsters to solve the task such that, delete an element in a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.					
Youngsters were lacking the idea to solve the problem.					
Being an exciting youngster can you solve it?					
Function Description					

1. Create a main function for the program.
2. In this main function accept the size and data of the queue by the user.
3. After that enqueue these element in the queue using an enqueue() function
4. Now to dequeue, we will be using an function that will remove elements from the queue one by one.
5. In this dequeue function first we will check if there are elements present in the queue or not.
6. If there are elements present in the queue then we will move the front of the queue by one step each time ,that is every time when an element is removed from the list.

Constraints:

$0 < \text{size} < 100$

$0 < \text{data} < 1000$

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 6 3 1 2

EXPECTED OUTPUT

Dequeuing elements:
6 3 1 2
3 1 2
1 2
2

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Dequeuing elements:
8 6 4 9 5 3 5
6 4 9 5 3 5
4 9 5 3 5
9 5 3 5
5 3 5
3 5
5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int data)

Test Case 2

KEYWORD

for (i=front;i<=rear;i++)

Test Case 3

KEYWORD

enqueue(data);

Test Case 4

KEYWORD

void dequeue()

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

Code:

```
1  #include <stdio.h>
2  #define MAX 1000
3
4  int queue[MAX];
5  int front = 0;
6  int rear = -1;
7
8  void enqueue(int value)
9  {
10     rear++;
11     queue[rear] = value;
12 }
13
14 int dequeue()
15 {
16     if (front > rear)
17         return -1;
18
19     return queue[front++];
20 }
21
22 void display()
23 {
24     for (int i = front; i <= rear; i++)
25     {
26         printf("%d", queue[i]);
27
28         if (i < rear)
29             printf(" ");
30     }
31
32     printf("\n");
33 }
34
35 int main()
36 {
37     int n, value;
```

```

38
39     scanf("%d", &n);
40
41     for (int i = 0; i < n; i++)
42     {
43         scanf("%d", &value);
44         enqueue(value);
45     }
46
47     printf("Dequeuing elements:\n");
48
49     while (front <= rear)
50     {
51         dequeue();
52
53         if (front <= rear)
54             display();
55     }
56
57     return 0;
58 }

```

Output:

Test Case-1:

```

5
5 6 3 1 2
Dequeuing elements:
6 3 1 2
3 1 2
1 2
2

-----
Process exited after 6.207 seconds with return value 0
Press any key to continue . . . |

```

Test Case-2:

```
8
1 8 6 4 9 5 3 5
Dequeuing elements:
8 6 4 9 5 3 5
6 4 9 5 3 5
4 9 5 3 5
9 5 3 5
5 3 5
3 5
5

-----
Process exited after 21.85 seconds with return value 0
Press any key to continue . . . |
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236